

MANUAL DE ENGENHARIA & ARQUITETURA DE SOFTWARE — ECOSSISTEMA NOVA

Dossiê Técnico, Arquitetural e Didático de Consolidação Profissional

Autor: Fábio Rodrigues | Ecossistema NOVA | Trilha Santander 2026 AI Java Back-end (DIO)

Sumário Executivo

Este compêndio consolida os fundamentos teóricos, padrões arquiteturais, código de produção e tecnologias emergentes implementadas no ecossistema **NOVA**. Ele serve como fonte definitiva de consulta para entrevistas técnicas, design de microsserviços e aceleração de estudos em **Java 21 LTS**, **Spring Boot 3**, **Clean Architecture**, **TDD**, **Bancos Relacionais** e **Agentes de Inteligência Artificial (MCP / Voice AI)**.

1. Java 21 LTS Moderno & Recursos Avançados

O Java 21 é uma versão de suporte de longo prazo (LTS) que consolida a transição do Java para uma linguagem mais expressiva, imutável e eficiente.

1.1. Records: Imutabilidade e Dados Puros

Records são classes imutáveis transparentes cujo propósito exclusivo é carregar dados. O compilador gera automaticamente os campos **private final**, construtor canônico, **getters** (sem o prefixo **get**), **equals()**, **hashCode()** e **toString()**.

```
// DTO imutável e conciso
public record TransacaoRequest(
    String descricao,
    BigDecimal valor,
    TipoTransacao tipo,
    CategoriaTransacao categoria,
    LocalDate data
) {
    // Compact Constructor para validações de domínio
    public TransacaoRequest {
        if (valor == null || valor.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("O valor da transação deve ser positivo.");
        }
        if (data == null) {
            data = LocalDate.now();
        }
    }
}
```

1.2. Pattern Matching (instanceof & Switch Expressions)

Elimina o casting manual redundante e torna a tomada de decisão tipada e exaustiva.

```
// Pattern Matching com Switch Expression
public String formatarRelatorio(Object obj) {
    return switch (obj) {
        case Transacao t when t.getValor().compareTo(new BigDecimal("1000")) > 0 ->
            "Transação de Alto Valor: " + t.getDescricao();
        case Transacao t ->
            "Transação Regular: " + t.getDescricao() + " | R$ " + t.getValor();
        case ResumoFinanceiro r ->
            "Balanço Consolidado: Saldo R$ " + r.saldo();
        case null -> "Dado nulo fornecido";
        default -> "Objeto não mapeado: " + obj.getClass().getSimpleName();
    };
}
```

1.3. Sealed Classes e Interfaces

Permitem restringir quais classes ou records podem estender ou implementar uma interface, criando hierarquias fechadas e controladas pelo compilador.

```
public sealed interface EventoFinanceiro permits TransacaoCriada, TransacaoCancelada, BalancoCalculado {}

public record TransacaoCriada(Long id, BigDecimal valor) implements EventoFinanceiro {}
public record TransacaoCancelada(Long id, String motivo) implements EventoFinanceiro {}
public record BalancoCalculado(BigDecimal saldo) implements EventoFinanceiro {}
```

1.4. Streams API & Processamento Funcional

A Streams API permite operações declarativas sobre coleções de dados, com filtros, transformações e agregações matemáticas de alta performance.

```
// Cálculo do total de despesas por categoria usando Streams
Map<CategoriaTransacao, BigDecimal> totalPorCategoria = transacoes.stream()
    .filter(t -> t.getTipo() == TipoTransacao.DESPESA)
    .collect(Collectors.groupingBy(
        Transacao::getCategoria,
        Collectors.reducing(BigDecimal.ZERO, Transacao::getValor, BigDecimal::add)
    ));
```

1.5. Concorrência Moderna & Virtual Threads (Project Loom)

O Java 21 introduziu os **Virtual Threads** (threads leves gerenciadas pela JVM e não pelo Sistema Operacional). Enquanto uma thread de plataforma do SO consome ~1MB de memória e tem criação cara, milhões de Virtual Threads podem coexistir consumindo poucos bytes, ideal para microsserviços I/O-bound com requisições HTTP e chamadas a bancos de dados.

2. Spring Boot 3.3+ & Frameworks Corporativos

2.1. Inversão de Controle (IoC) & Injeção de Dependências (DI)

O Spring gerencia o ciclo de vida dos componentes através do seu **ApplicationContext**. A melhor prática moderna é a injeção via construtor com campos **final**, garantindo imutabilidade e facilidade de teste unitário sem necessidade de subir o contexto Spring.

```
@Service
public class CadastrarTransacaoUseCase {

    private final TransacaoRepository repository; // Porta de Domínio

    // Injeção de dependência via construtor (sem @Autowired explícito)
    public CadastrarTransacaoUseCase(TransacaoRepository repository) {
        this.repository = repository;
    }

    public TransacaoResponse executar(TransacaoRequest request) {
        Transacao transacao = new Transacao(
            request.descricao(),
            request.valor(),
            request.tipo(),
            request.categoria(),
            request.data()
        );
        Transacao salva = repository.salvar(transacao);
        return TransacaoResponse.fromDomain(salva);
    }
}
```

2.2. Spring Data JPA & Derived Queries

Abstrai operações de banco de dados eliminando código boilerplate de JDBC ou EntityManager.

```
public interface SpringDataTransacaoRepository extends JpaRepository<TransacaoEntity, Long> {
    List<TransacaoEntity> findByDataBetween(LocalDate inicio, LocalDate fim);
    List<TransacaoEntity> findByTipo(TipoTransacao tipo);
}
```

2.3. APIs RESTful & RFC 7807 ProblemDetails

O padrão moderno de resposta de erro HTTP exige estrutura semântica detalhada para que clientes e agentes inteligentes entendam exatamente a falha.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BusinessException.class)
    public ProblemDetail handleBusinessException(BusinessException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.UNPROCESSABLE_ENTITY, ex.getMessage()
        );
        problem.setTitle("Regra de Negócio Violada");
        problem.setProperty("timestamp", Instant.now());
        return problem;
    }
}
```

2.4. Spring AI & Model Context Protocol (MCP)

O **Model Context Protocol (MCP)** é o padrão aberto que permite que Modelos de Linguagem (LLMs) executem ferramentas corporativas de forma autônoma e segura. No Spring Boot, anotamos métodos com **@Tool**:

```
@Component
public class FinanceiroMcpTools {

    private final CadastrarTransacaoUseCase cadastrarUseCase;
    private final CalcularResumoFinanceiroUseCase resumoUseCase;

    public FinanceiroMcpTools(CadastrarTransacaoUseCase c, CalcularResumoFinanceiroUseCase r) {
        this.cadastrarUseCase = c;
        this.resumoUseCase = r;
    }

    @Tool(description = "Registra uma nova transação financeira no banco de dados persistente.")
    public TransacaoResponse cadastrarTransacao(
        @ToolParam(description = "Descrição da despesa ou receita") String descricao,
        @ToolParam(description = "Valor monetário positivo") double valor,
        @ToolParam(description = "Tipo: RECEITA ou DESPESA") String tipo,
        @ToolParam(description = "Categoria contábil") String categoria,
        @ToolParam(description = "Data no formato AAAA-MM-DD") String data) {

        TransacaoRequest request = new TransacaoRequest(
            descricao,
            BigDecimal.valueOf(valor),
            TipoTransacao.valueOf(tipo.toUpperCase()),
            CategoriaTransacao.valueOf(categoria.toUpperCase()),
            LocalDate.parse(data)
        );
        return cadastrarUseCase.executar(request);
    }
}
```

3. Clean Architecture & Arquitetura Hexagonal (Ports & Adapters)

O objetivo central da Clean Architecture (Robert C. Martin) e da Arquitetura Hexagonal (Alistair Cockburn) é o **isolamento total das regras de negócio em relação a detalhes tecnológicos externos** (bancos de dados, frameworks web, UI, bibliotecas de terceiros).

3.1. A Estrutura em 3 Camadas no NOVA

```

+-----+
| INFRASTRUCTURE (Adaptadores Externos) |
| - Web Controllers (REST Endpoints)    |
| - Persistence Entities (JPA / Hibernate) |
| - Spring Data Repositories & H2 Database |
| - Spring AI MCP Server (@Tool)         |
|                                         |
| +-----+                             |
| | APPLICATION (Casos de Uso & Orquestracao) | | | |
| | - Use Cases (CadastrarTransacao, ListarTransacoes) | |
| | - DTOs (Records de Request/Response) | |
| |                                         | |
| | +-----+                             | |
| | | DOMAIN (Nucleo Puro de Negocio) | | |
| | | - Entidades de Dominio (Transacao, Balanco) | | |
| | | - Value Objects & Enums | | |
| | | - Interfaces de Repositorio (Ports) | | |
| | | - Regras e Invariantes de Negocio | | |
| | +-----+                             | |
| +-----+                             |
+-----+

```

3.2. A Regra de Ouro da Dependência

- As dependências de código-fonte apontam **SEMPRE DE FORA PARA DENTRO**.
- O **Domínio** não importa nenhuma classe do Spring, Hibernate, JPA ou Jackson.
- A **Aplicação** depende apenas do Domínio.
- A **Infraestrutura** implementa as interfaces do Domínio (Inversão de Dependência - DIP).

4. Testes Automatizados & Test-Driven Development (TDD)

4.1. O Ciclo TDD (Red-Green-Refactor)

1. **Red:** Escrever um teste que define o comportamento esperado antes de escrever o código. O teste falha.
2. **Green:** Escrever a menor quantidade de código possível para fazer o teste passar.
3. **Refactor:** Melhorar a estrutura do código, remover duplicações e aplicar padrões, garantindo que o teste continue verde.

4.2. JUnit 5 + Mockito + AssertJ na Prática

```
@ExtendWith(MockitoExtension.class)
@DisplayName("Testes do Caso de Uso: Cadastrar Transação")
class CadastrarTransacaoUseCaseTest {

    @Mock
    private TransacaoRepository repository; // Mock da porta de saída

    @InjectMocks
    private CadastrarTransacaoUseCase useCase;

    @Test
    @DisplayName("Deve cadastrar uma transação válida com sucesso")
    void deveCadastrarTransacaoValida() {
        // 1. Arrange (Preparação)
        TransacaoRequest request = new TransacaoRequest(
            "Aluguel",
            new BigDecimal("600.00"),
            TipoTransacao.DESPESA,
            CategoriaTransacao.MORADIA,
            LocalDate.of(2026, 8, 1)
        );

        when(repository.salvar(any(Transacao.class)))
            .thenAnswer(invocation -> {
                Transacao t = invocation.getArgument(0);
                t.setId(1L);
                return t;
            });

        // 2. Act (Execução)
        TransacaoResponse response = useCase.executar(request);

        // 3. Assert (Verificação com AssertJ)
        assertThat(response).isNotNull();
        assertThat(response.id()).isEqualTo(1L);
        assertThat(response.descricao()).isEqualTo("Aluguel");
        assertThat(response.valor()).isEqualByComparingTo("600.00");
        assertThat(response.tipo()).isEqualTo(TipoTransacao.DESPESA);

        // Verificação de comportamento com Mockito
        verify(repository, times(1)).salvar(any(Transacao.class));
    }
}
```

5. Bancos de Dados Relacionais & Persistência ACID

5.1. O que são Garantias ACID?

- **A — Atomicidade:** Uma transação é tratada como uma unidade indivisível. Todas as operações são confirmadas (**commit**) ou revertidas (**rollback**).
- **C — Consistência:** A transação leva o banco de um estado válido a outro estado válido, respeitando chaves primárias, constraints e regras de integridade.
- **I — Isolamento:** Transações concorrentes executam sem interferir no estado intermediário umas das outras.
- **D — Durabilidade:** Uma vez confirmada a transação, os dados permanecem gravados no disco mesmo diante de falhas de energia ou reinicialização.

5.2. Persistência em Arquivo Físico com H2

Ao contrário de bancos em memória (**jdbc:h2:mem:...**) que perdem dados ao encerrar o processo, o **NOVA** utiliza persistência em arquivo binário (**mv.db**):

```
spring:
  datasource:
    url: jdbc:h2:file:./data/financiadb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE;AUTO_SERVER=TRUE
    driverClassName: org.h2.Driver
    username: sa
    password:
  jpa:
    database-platform: org.hibernate.dialect.H2Dialect
    hibernate:
      ddl-auto: update
```

6. Inteligência Artificial & Agentes Autônomos

6.1. Model Context Protocol (MCP)

Criado para padronizar como LLMs acessam bases de dados e ferramentas computacionais. Ele opera sobre transporte **SSE (Server-Sent Events)** ou **stdio**:

1. O Cliente MCP (ex: Antigravity / Claude) inicia o handshake e descobre as ferramentas (**tools/list**).
2. O Servidor MCP (Spring Boot) responde com o esquema JSON de parâmetros de cada **@Tool**.
3. O LLM decide chamar a ferramenta e emite um **tools/call**.
4. O servidor executa a regra de negócio no banco H2 e devolve o resultado estruturado.

6.2. Arquitetura de Agentes de Voz em Tempo Real (Sofia Voice AI)

O pipeline de voz de baixa latência conecta múltiplos serviços assíncronos:

```
[Usuario / Telefone]
  | (Audio PCM / WebRTC)
  v
[Vapi Platform] ----> [STT: Whisper / Deepgram] (Audio -> Texto ~120ms)
  |
  v
[Orquestrador de IA] ----> [LLM: OpenAI GPT-4o-Mini + RAG] (Inferencia ~250ms)
  |
  v
[TTS: Cartesia / Clara V2] (Texto -> Audio Neural com Pausas ~150ms)
  |
  v
[Webhook Assíncrono] ----> [WhatsApp API / CRM / NOVA Backend] (Acao & Persistencia)
```

7. Python, Automação & Geração Documental

7.1. Matplotlib e o Motor de Gráficos (chart_engine.py)

- Utiliza o backend sem interface gráfica **matplotlib.use('Agg')** para execução segura em servidores.
- Renderiza gráficos com densidade de 300 DPI, paletas de cores corporativas (Navy **#1A2530**, Azul **#2980B9**, Verde **#27AE60**) e layout proporcional (**tight_layout()**).

7.2. ReportLab e python-docx

- **ReportLab**: Constrói árvores de elementos visuais (**Flowables**) calculando quebras de página automáticas, margens de precisão milimétrica e estilos tipográficos imutáveis para garantir PDFs 100% legíveis por ATS.
- **python-docx**: Manipula a estrutura XML padrão OpenXML da Microsoft (**w:pBdr**, **w:r**, **w:p**), injetando cabeçalhos timbrados e tabelas formais para exportação profissional.

Conclusão & Próximos Passos

Este compêndio serve como base de engenharia de software para a consolidação profissional do desenvolvedor. A integração prática desses conceitos no ecossistema **NOVA** demonstra prontidão técnica e capacidade de liderança no desenvolvimento de microserviços modernos e IA aplicada.